Partie 6: Couche services – Patrons de conceptions

6.1. Implémentation du patron Repository

1. Sous le dossier **Interfaces**, ajouter une interface générique nommée **IGenericRepository**. Cette dernière contient toutes les méthodes CRUD.

```
namespace AM.ApplicationCore.Interfaces
{
    public interface IGenericRepository<TEntity> where TEntity : class
    {
        void Add(TEntity entity);
        void Update(TEntity entity);
        void Delete(TEntity entity);
        void Delete(Expression<Func<TEntity, bool>> where);
        TEntity GetById(params object[] keyValues);
        TEntity Get(Expression<Func<TEntity, bool>> where);
        IEnumerable<TEntity> GetAll();
        IEnumerable<TEntity> GetMany(Expression<Func<TEntity, bool>> where);
    }
}
```

2. Sous le projet **AM.Infrastructure**, créer une classe nommée **GenericRepository.** Cette dernière implémente l'interface **IGenericRepository**.

```
using AM.ApplicationCore.Interfaces;
using Microsoft.EntityFrameworkCore;
using System.Linq;
using System.Linq.Expressions;

namespace AM.Infrastructure
{
    public class GenericRepository<T> :
IGenericRepository<T> where T : class
    {
        private readonly DbContext _context;
        private readonly DbSet<T> _dbSet;
        private readonly IUnitOfWork
_unitOfWork;
        public GenericRepository(DbContext
context)
        {
            _context = context;
            _dbSet = _context.Set<T>();
        }
        public void Add(T entity)
        {
            _dbSet.Add(entity);
        }
        public void Delete(T entity)
        {
            _dbSet.Remove(entity);
        }
        public void Delete(Expression<Func<T,
bool>> where)
        {
_dbSet.RemoveRange(_dbSet.Where(where));
        }
        public T Get(Expression<Func<T, bool>>
where)
        {
            return
_dbSet.Where(where).FirstOrDefault();
        }

        public IEnumerable<T> GetAll()
        {
            return _dbSet.AsEnumerable();
        }
        public T GetById(params object[]
keyValues)
        {
            return _dbSet.Find(keyValues);
        }
        public IEnumerable<T>
GetMany(Expression<Func<T, bool>> where)
        {
            return _dbSet.Where(where)
                .AsEnumerable();
        }
        public void Update(T entity)
        {
            _dbSet.Update(entity);
        }
    }
}
```

3. Sous le dossier **Interfaces** du projet **AM.ApplicationCore**,

   - Créer l'interface de services **IServicePlane** contenant les trois méthodes suivantes :

     - Une méthode **Add()** qui permet d'ajouter dans la BD un avion.
     - Une méthode **Remove()** qui permet de supprimer de la BD un avion.
     - Une méthode **GetAll()** qui permet de retourner tous les avions enregistrés dans la BD.

```
namespace AM.ApplicationCore.Interfaces
{
    public interface IServicePlane
    {
        public void Add(Plane plane);
        public void Remove(Plane plane);
        public IList<Plane> GetAll();
    }
}
```

   - Ajouter à l'interface **IServiceFlight** les trois méthodes précédentes.

```
namespace AM.ApplicationCore.Interfaces
{
   public interface IServiceFlight
    {
        public void Add(Flight flight);
        public void Remove(Flight flight);
        public IList<Flight> GetAll();
        . . .
    }
}
```

4. Sous le dossier **Services**,

   - Créer la classe **ServicePlane** qui implémente l'interface **IServicePlane** et qui prend en paramètre du constructeur un objet de type **IRepositoryGeneric**.

```
namespace AM.ApplicationCore.Services
{
    public class ServicePlane : IServicePlane
    {
        private readonly IGenericRepository<Plane> _repo;

        public ServicePlane(IGenericRepository<Plane> repo)
        {
            _repo = repo;
        }
        public void Add(Plane plane)
        {
            _repo.Add(plane);
       }
       public IList<Plane> GetAll()
       {
            return _repo.Planes.ToList();
        }

        public void Remove(Plane plane)
        {
            _repo.Remove(plane);
        }
    }
}
```

- Modifier la classe **ServiceFlight** afin d'implémenter l'interface **IServiceFlight en** prenant en constructeur un objet de type **IRepositoryGeneric**.

```
namespace AM.ApplicationCore.Services
{
    public class ServiceFlight : IServiceFlight
    {
        private readonly IGenericRepository<Flight> _repo;
        public ServiceFlight(IGenericRepository<Flight> repo)
        {
            _repo = repo;
        }
        public void Add(Flight flight)
        {
            _repo.Add(flight);
        }
       public IList<Flight> GetAll()
       {
            return _repo.Flights.ToList();
        }

        public void Remove(Flight flight)
        {
            _repo.Remove(flight);
        }
        . . .
    }
}
```

5. Que constatez-vous ?

   Pas de la persistance de données.

   Impossible d'utiliser la classe du contexte dans le projet **AM.ApplicationCore.**

   Solution : Utilisation du parton UnitOfWork

6.2. Implémentation du patron UnitOfWork

1. Sous le dossier **Interfaces**, ajouter une interface nommée **IUnitOfWork** contenant une **propriété** de type DbContext et une méthode **Save()**.

```
namespace AM.ApplicationCore.Interfaces
{
    public interface IUnitOfWork : IDisposable
    {
        int Save();
        IGenericRepository<TEntity> Repository<TEntity>() where TEntity : class;
    }
}
```

2. Dans le projet **AM.Infrastructure**, créer une classe nommée **UnitOfWork** qui implémente l'interface **IUnitOfWork**.

```
namespace AM.Infrastructure
{
    public class UnitOfWork : IUnitOfWork
    {
        private readonly DbContext _context;
        private readonly Type _repositoryType;
        private bool disposedValue;

        public UnitOfWork(DbContext context, Type repositoryType)
        {
            _context = context;
            _repositoryType = repositoryType;
        }
        public IGenericRepository<T> Repository<T>() where T : class
        {
            return (IGenericRepository<T>)Activator.CreateInstance(_repositoryType
                        .MakeGenericType(typeof(T)), _context);
        }
        public int Save()
        {
            return _context.SaveChanges();
        }
    }
}
```

3. Modifier les deux classes **ServicePlane** et **ServiceFlight** afin d'utiliser un objet de type **IUnitOfWork**.

```
namespace AM.ApplicationCore.Services
{
    public class ServicePlane : IServicePlane
    {
    private IUnitOfWork unitOfWork;
    public ServicePlane(IUnitOfWork
unitOfWork)
    {
        this.unitOfWork = unitOfWork;
    }

    public void Add(Plane p)
    {
        unitOfWork.Repository<Plane>().Add(p);
    }

    public void Update(Plane p)
    {

unitOfWork.Repository<Plane>().Update(p);
    }

    public IList<Plane> GetAll()
    {
        return
unitOfWork.Repository<Plane>().GetAll().ToList
();
    }
}
```

```
namespace AM.ApplicationCore.Services
{
    public class ServiceFlight : IServiceFlight
    {
    private IUnitOfWork unitOfWork;
    public ServiceFlight(IUnitOfWork
unitOfWork)
    {
        this.unitOfWork = unitOfWork;
    }

    public void Add(Flight p)
    {

unitOfWork.Repository<Flight>().Add(p);
    }

    public void Update(Flight p)
    {

unitOfWork.Repository<Flight>().Update(p);
    }

    public IList<Flight> GetAll()
    {
        return
unitOfWork.Repository<Flight>().GetAll().ToLis
t();
    }
}
```

4. Que constatez-vous ?

   Le garbage collector (GC) ne libère pas les objets de type DbContext. Ce dernier est un objet non géré par GC.

   Solution : Utilisation de l'interface **IDisposable**

5. Modifier l'interface **IUnitOfWork** afin qu'elle hérite de l'interface **IDisposable**.

```
namespace AM.ApplicationCore.Interfaces
{
    public interface IUnitOfWork : IDisposable
    {
        int Save();
        IGenericRepository<TEntity> Repository<TEntity>() where TEntity : class;
    }
}
```

6. Modifier la classe **UnitOfWork** en implémenter la méthode Dispose (bool disposing). Cette méthode permet de libérer l'objet du **contexte**.

```
namespace AM.Infrastructure
{
    public class UnitOfWork : IUnitOfWork
    {
        . . .
        private bool disposedValue;

        protected virtual void Dispose(bool
disposing)
        {
            if (!disposedValue)
            {
                if (disposing)
                {
                    _context.Dispose();
                }
                disposedValue = true;
            }
        }
        ~UnitOfWork()
        {
            Dispose(disposing: false);
        }

        public void Dispose()
        {
            Dispose(disposing: true);
            GC.SuppressFinalize(this);
        }
    }
}
```